Detection, Mitigation, and Triage Guide

Audience: an L1 analyst who has the alert in front of them, and an L2 or detection engineer who owns the rules.

This guide uses INC-001 as the worked example. Rule IDs are Wazuh 4.14.7 stock unless marked custom.

---

1. Detection model

The incident produced four kinds of evidence. Each answers a different question.

Layer	Source	Question it answers	INC-001 signal
Identity	sshd / journald	Who tried to log in, and did it work?	5760, 5763, 5715, 40112
Process	auditd execve	What ran after the login?	80792 + decoded argv
File	FIM realtime	What changed on disk?	100120, `authorized_keys`
Network	Suricata	What left the host?	None for SSH; a `curl` in process logs

A single layer is never enough. The identity layer says "maybe"; the process and file layers say "confirmed".

1.1 Signal hierarchy

```
Low confidence   5760  one failed password
                 5715  one successful login
                 ----  noise; do not open a ticket per event

Medium           5763  brute force (8 failures / 120 s)
                 40112 failures followed by a success
                 ----  open a ticket, investigate

High             100120 authorized_keys added
                 new account, sudoers change, cron/systemd unit created
                 ----  treat as compromise until proven otherwise
```

---

2. Rule-by-rule

2.1 Rule 5760 — `sshd: authentication failed`

- **Level:** 5
- **Fires:** one failed password or keyboard-interactive attempt.
- **Fields:** `srcip`, `srcport`, `srcuser`.
- **Use:** volume and account context. A single one is a typo. Twenty in 30 seconds is an attack.
- **Limitation:** no notion of the account's importance, and no threshold.

2.2 Rule 5763 — `sshd: brute force trying to get access to the system`

- **Level:** 10
- **Fires:** 8 failures in 120 seconds from the same source (`frequency` / `timeframe`).
- **INC-001:** fired at 15:22:01, 11 seconds after the first failure.
- **Use:** the first genuinely actionable alert. It is the anchor for the rest of the investigation.
- **Limitation:** source-IP based. Slow sprays below 8/120 s, or distributed sources, evade it.

2.3 Rule 40112 — `Multiple authentication failures followed by a success`

- **Level:** 12
- **Fires:** any `authentication_success` after any `authentication_failures` from the same source, within 240 seconds.
- **INC-001:** fired twice on the attacker (15:22:21, 15:22:23) and six times on four routine analyst sessions.
- **Why the false positives:** the rule is account-blind and authentication-method-blind. Every session shared `10.0.2.2`.
- **Use:** a candidate, not a verdict. Always confirm the account and the method.

2.4 Rule 100120 — custom, FIM persistence

```
<rule id="100120" level="12">
  <if_sid>550,554</if_sid>
  <match>authorized_keys</match>
  <description>FIM: SSH authorized_keys added or modified (possible persistence)</description>
  <mitre>
    <id>T1098.004</id>
    <id>T1078</id>
  </mitre>
  <group>syscheck,persistence,</group>
</rule>
```

- **Parent rules:** 550 (checksum changed), 554 (file added). Both are FIM events.
- **Match:** the literal file name, so it covers every user's key file.
- **INC-001:** fired one second after the write, with path, owner, and SHA-256.
- **False positive:** legitimate key rotation, onboarding, a user adding a second device.
- **Limitation:** name-based. An attacker who writes the file under a different name, or a tool that bypasses FIM, is not covered. Pair with authentication logs and key fingerprint inventory.

Test:

```
sudo -u svc-backup bash -c \
  'echo "ssh-ed25519 AAAATEST" >> ~/.ssh/authorized_keys'
# expect 100120 within a second, with syscheck.path and sha256_after
```

---

3. Tuning

3.1 Narrow 40112

Stock definition (Wazuh 4.14.7, `0280-attack_rules.xml`):

```
<rule id="40112" level="12" timeframe="240">
  <if_group>authentication_success</if_group>
  <if_matched_group>authentication_failures</if_matched_group>
  <same_source_ip />
  <description>Multiple authentication failures followed by a success.</description>
</rule>
```

Target behaviour:

- only SSH authentication events, not every PAM/sudo failure;
- a minimum of 10 failures, not one;
- the success must be password-based;
- ideally, the same account as the failures.

The account condition is the difficult one. Wazuh exposes `srcuser` on the failure rules and `dstuser` on the success rule, and `<same_user />` does not bridge them. Do not ship a rule that silently never fires (see `rules/tuning-notes.md`).

Options, in order of preference:

1. **Tune 40112 itself** with `overwrite="yes"` in `local_rules.xml`, narrowing groups and raising the threshold. Change-controlled and tested in the lab.
2. **Add an enrichment step**: a small decoder or script that copies `srcuser` into a common field, then correlate on that field.
3. **Accept source-IP correlation** and rely on the analyst to check the account. This is what the stock rule does; document the false-positive rate.

3.2 Reduce auditd noise

INC-001 produced 292 execve alerts in 150 seconds. That is collection, not detection.

- Audit only command categories that matter (shells, `curl` / `wget`, `useradd`, `crontab`, `systemctl`, `sudo`), not every binary.
- Or keep full execve but aggregate per session and alert on sequences, not individual commands.
- Filter the investigation, not the collection: when triaging, pivot on `uid` or `audit.session`, not on the whole host.

3.3 Baseline the key files

```
find /home -path '*/.ssh/authorized_keys' -type f -exec sha256sum {} + \
  | sort > baseline-authorized-keys.txt
```

Store the baseline in version control or a SIEM lookup. Alert on any hash not in it. This turns 100120 from "a file changed" into "an unauthorised key appeared".

3.4 Add a service-account login rule

The account should never open a shell. A high-value, low-noise rule:

```
<rule id="100121" level="10">
  <if_sid>5715</if_sid>
  <field name="dstuser">^svc-</field>
  <description>SSH: interactive login by a service account ($(dstuser))</description>
  <mitre><id>T1078</id></mitre>
</rule>
```

Test the field matching before deploying; Wazuh's field matching is exact by default in some versions and regex in others. If regex is unsupported, enumerate the accounts.

---

4. Triage playbook

Target: reach a defensible verdict in five minutes.

4.1 The five questions

1. **Did it succeed?** Look for 5715, 40112, or `Accepted` in the auth log.
2. **Which account?** Is it human, service, privileged, shared?
3. **What ran after?** auditd execve around the success time, filtered by `uid`.
4. **What changed?** FIM events for that user: `authorized_keys`, cron, systemd, sudoers.
5. **Where else?** Same account, key, or source on other hosts.

4.2 Decision tree

```
Alert: brute force or failures followed by success
│
├─ No success found
│   ├─ Failures only, one account, low volume → close as attempted, tune threshold
│   └─ High volume or many accounts → block source, watch for success, ticket Low/Medium
│
└─ Success found
    ├─ Same account, password auth, preceded by failures → likely compromise
    │   ├─ Post-login commands present → confirm scope, escalate High
    │   ├─ FIM key/cron/systemd change → persistence, escalate High/Critical
    │   └─ sudo/root obtained → Critical
    └─ Different account, key auth, known admin → false positive; document why
```

4.3 Step by step

1. **Read the rule and the raw line.** Do not trust the description alone.

2. **Establish the timeline.** First failure, last failure, first success.
3. **Confirm the method.** `Accepted password` is very different from `Accepted publickey`.
4. **Pivot to process telemetry.** Filter auditd by the account or session, not the whole host.
5. **Check persistence, in this order:** `authorized_keys`, `cron` / `systemd` timers, new local accounts, `sudoers`, shell profile files.
6. **Check egress.** A `curl` / `wget` after login is a data-flow question, not a curiosity.
7. **Assign severity** with the matrix in §5.
8. **Preserve before remediating.** Hash the key file, export the alerts, keep the audit records.
9. **Contain** with the least destructive action that stops the access.
10. **Escalate** with the template in §7.

4.4 What "good" looks like

A ticket that lets L2 act without re-investigating:

- exact timestamps, UTC;
- account, source, host;
- method (password/key);
- post-login commands, preferably decoded;
- persistence artefacts with hashes;
- whether privilege escalation succeeded;
- the containment already taken;
- the specific question for L2.

---

5. Severity matrix

Outcome	No persistence	Persistence	Root / new admin
Failed attempts only	Low	Low	Medium
Unprivileged account, no post-login activity	Medium	High	Critical
Unprivileged account, discovery/egress	High	High	Critical
Privileged account	High	Critical	Critical
Lateral movement observed	Critical	Critical	Critical

INC-001: unprivileged account, discovery, persistence, no root → **High**.

Adjust for asset criticality and data sensitivity. A domain controller raises every cell by one level.

---

6. Containment and mitigation

6.1 During the incident

Action	How	Caution
Isolate the host	EDR network containment, or lab: disconnect	Preserve memory/disk if forensics is in scope
Disable the account	`usermod -L`, `chage -E 0`	Check for service impact first
Kill sessions	`pkill -KILL -u svc-backup`	Do this after capturing the process list
Remove the key	Edit `authorized_keys`	Record the SHA-256 first
Reset credentials	`passwd`, revoke tokens	The key must be removed too, or the reset is cosmetic
Block the source	Firewall / fail2ban	Only if the source is not a shared NAT or jump host

6.2 Host hardening

```
# service accounts should not have shells
sudo usermod -s /usr/sbin/nologin svc-backup

# global password-auth policy (only if keys are universally deployed)
sudo sed -i 's/^#\?PasswordAuthentication.*/PasswordAuthentication no/' \
  /etc/ssh/sshd_config

# reduce guessing budget
sudo sed -i 's/^#\?MaxAuthTries.*/MaxAuthTries 3/' /etc/ssh/sshd_config
sudo sed -i 's/^#\?LoginGraceTime.*/LoginGraceTime 20/' /etc/ssh/sshd_config
sudo sshd -t && sudo systemctl reload ssh
```

Lockout with `pam_faillock` (Debian path shown; verify before deploying):

```
sudo apt-get install -y libpam-modules
# /etc/pam.d/common-auth
# auth    required    pam_faillock.so preauth
# auth    [default=die] pam_faillock.so authfail
# account required    pam_faillock.so
sudo faillock --user svc-backup
```

6.3 Policy controls

- **Keys only** for non-human accounts; no password, no interactive shell.
- **MFA** for interactive SSH, or a bastion that enforces it.
- **Key inventory**: fingerprint, owner, purpose, expiry.
- **Least privilege**: service accounts should not be in `sudo` groups.
- **Management network**: SSH reachable only from a jump host, not the general estate.
- **Change control** for `authorized_keys`; legitimate rotation is the main false positive.

6.4 Detection controls

Control	Purpose
100120 + key hash baseline	Catch unauthorised persistence
Service-account login rule	Catch policy violation at first login
40112 tuning	Reduce false positives
Auditd sequence rules	Catch discovery → persistence → egress
Weekly key inventory diff	Catch slow, low-volume additions

---

7. Escalation template

```
INC-001 — SSH brute force, successful login, persistence

What:  20 failed passwords for svc-backup between 15:21:50 and 15:22:19 UTC,
       followed by a successful password login at 15:22:21.
Scope: Host soc-victim (10.0.2.15), account svc-backup (uid 1002), source 10.0.2.2.
Post:  Discovery (id, whoami, uname), sudo attempted and failed, SSH key added to
       /home/svc-backup/.ssh/authorized_keys at 15:22:22
       SHA-256 61954205be034570c75bdd6db67b3fbd3410ac1d4bebb08b65d117667965c5e8.
Done:  Host isolated. Account disabled. Key removed after hashing. Evidence preserved.
Ask:   (1) estate-wide search for the key fingerprint; (2) confirm no other host has
       password auth on service accounts; (3) decide whether to force a key rotation
       for all users on this host.
```

---

8. Post-incident

- Add the attack to the detection test suite: 20 failures + success must produce 5763, 40112 (or its tuned replacement), and 100120.
- Measure: time from first failure to 5763 (11 s), from key write to 100120 (<1 s).
- Review whether the account needed a shell at all. If not, the incident is a policy finding, not just a technical one.
- Update the runbook with the field-model limitation (`srcuser` vs `dstuser`).
- Re-baseline `authorized_keys` hashes after remediation.

---

9. Common mistakes

Mistake	Consequence
Closing 40112 as a false positive because the user is known	The account, not the user, is the unit of analysis
Resetting the password and stopping	The key remains valid
Blocking the source IP	Shared NAT or a jump host makes this harmful and ineffective
Treating 5760 volume as the incident	The success is the incident; failures are precursor
Trusting the rule description without the raw log	Rule text is generic; the account and method decide
Remediating before hashing	Evidence is destroyed
Alerting on every execve	292 alerts in 150 seconds; the real one is buried

10. Detection coverage map

Technique	ID	Covered by	Gap
Password guessing	T1110.001	5760, 5763	Slow/distributed sprays
Valid accounts	T1078	5715, 40112	Method/account context
SSH authorized keys	T1098.004	100120	Non-standard paths; FIM bypass
Unix shell	T1059.004	80792	Noise; argv truncation
System discovery	T1082, T1033	80792	Needs sequence logic
Remote services	T1021.004	5715	Cannot distinguish admin from attacker
Exfiltration over HTTP	T1041	Suricata (if enabled)	Encrypted egress; no proxy logs

11. Metrics

Metric	INC-001 value
First failure → brute-force alert (5763)	11 s
Success → persistence alert (100120)	<1 s
Alerts in window	400
Signal-to-noise	1 true compromise / 8 high-level auth alerts
Time to triage (exercise)	~4 minutes
Containment actions	5

A five-minute triage objective is achievable here because the evidence is already correlated. The bottleneck is deciding which of the 40112 hits is real, not collecting data.